On-call incident: API error rate 0.2 % → 7 %

Investigation plan, escalation decision at T+30 and three preventive improvements

Alert: 14:02 — API error rate jumps from 0.2 % to 7 %

Signals: 2 of 5 instances at 95 % CPU; no deploy in 24 h; normal traffic

Customer: One enterprise customer reports intermittent 502s

Logs: 502 after ~5.1–5.3 s; retries; pool 100 active / 0 idle; 4,800 ms query; /api/health 200 in 8 ms

Path: API → internal service → database

Irakli — L2 Support Engineer candidate · 18 October 2026

1. Summary

My reading of the evidence before touching anything: the API is not broken on its own. The 502s land just after a 5-second upstream timeout, which means the API is giving up on something downstream. The internal service's database pool is fully used with nothing idle, and a query that should take milliseconds is taking 4.8 seconds. A slow query holding connections, amplified by retries, is the most likely story. The green health check is not evidence of health, because it never touches the database.

Placed on the call path, the symptoms line up. The client sees 502s after 5.1–5.3 s. At the API, two of five instances run at 95 % CPU while /api/health answers 200 in 8 ms without touching the database. The internal service retries, its pool shows 100 active and 0 idle connections and logs 'context deadline exceeded'. The database has a query taking 4,800 ms. The timeouts surface at the API, but the waiting happens downstream.

2. Investigation plan

2.1 First five minutes: golden signals per tier

I start with the four golden signals (errors, latency, traffic, saturation) for each tier, because a table like this shows within minutes where the time is going, and it is the same table I will paste into the escalation.

Signal	API	Internal service	Database
Errors	5xx by endpoint, instance, tenant	Error and timeout rate	Failed or cancelled statements
Latency	p50/p95/p99 vs the 5 s timeout	p95/p99; queue wait for a DB connection	Slow-query log 14:01–14:02; lock waits
Traffic	Client req/s (normal)	Req/s including retries (up to 2× client)	Active sessions, statements/s
Saturation	CPU 95 % on 2 of 5; threads	Pool 100/100/0; threads; GC	CPU/IO; connections vs max; replica lag

2.2 The steps

The steps overlap rather than run strictly one after another: scoping runs from minute 2 to 8, walking the chain from 8 to 20, mitigation from about 12 to 25, and from minute 15 I start preparing the escalation so it is ready at T+30.

Step 0 (minutes 0–2): acknowledge. Confirm the alert and the customer report describe the same thing (502s on /api/orders, not a dashboard artefact), note the exact start time from the metric and open an incident channel with a running timeline. Everything later, including the escalation, hangs off that start time.

Step 1 (minutes 2–8): scope before touching anything. Break errors down by endpoint, instance, tenant and status code before changing anything. If the 502s concentrate on the two hot instances, I have two bad instances. If they are spread evenly, a shared dependency is slow and the hot instances are simply carrying the retries. I also ask what changed at 14:02 even without a deploy: a config or feature-flag push, a cron job or import, certificate rotation, autoscaling, cloud maintenance, a database failover or a query-plan change.

Step 2 (minutes 8–20): walk the chain from the outside in. At the API: timeouts, retry configuration and retry counts. At the internal service: request rate compared with the client rate (retries can double it while traffic 'looks normal'), connection wait time and pool metrics. At the database: locks, long-running statements, the 4.8-second query, and any plan or statistics change that makes an old query slow without code changing.

Why one slow query is enough. A pool can serve at most its size divided by how long each query holds a connection (Little's law). With 100 connections and 50 ms queries that is about 2,000 queries per second; at 4.8 s it drops to about 21. A handful of slow queries therefore starves every other request of a connection, which is why one slow query is enough to explain the whole picture. These numbers are illustrative arithmetic from the scenario, not measurements.

2.3 Ranked hypotheses

#	Hypothesis	Confirms it	Rejects it
1	Slow query exhausting the pool	Pool = 100 lines up with slow-query entries and lock waits; killing the query drops errors within seconds	Pool saturated while DB latency is normal
2	Retry amplification	Internal request rate ⩾ client rate; CPU tracks retry volume	Retry count flat at 14:02
3	Two degraded or over-weighted instances	Unequal per-instance request counts; draining them fixes it	Errors spread across all five
4	One tenant's expensive pattern	Slow-query parameters belong to one account	Many tenants' parameters
5	Silent change or infrastructure	Config, index, failover or throttling event at 14:02	Nothing in config/DB/cloud history

'No deployment' rules out application code, not configuration or data. The complaining enterprise customer may turn out to be the cause as well as the victim (hypothesis 4).

2.4 Quick mitigations and their risks

Mitigation	Expected effect	Risk
Drain/restart the two hot instances, one at a time	Relief if the instances are the problem	If the DB is the cause, load just moves
Cap retries (one, jittered, idempotent reads only)	Immediate relief from amplification	Slightly higher visible error rate
Kill the long query or block the pattern/tenant	Often fixes it	May abort a legitimate job; record it and tell the owner
Raise pool size or timeout	Only with clear DB headroom	Bigger pool loads a slow DB more; longer timeout hides the problem
Rate-limit the expensive endpoint for one tenant	Protects other customers	That tenant sees more errors

Quantifying impact. Failed requests per minute = error rate × request rate; number of tenants above 1 % errors; the enterprise customer's failure share; duration so far; SLA budget burned. Every mitigation gets a timestamp and an observed effect in the timeline.

3. Escalation decision at T+30

Thirty minutes in, the error rate still swings between 5 and 8 %, CPU is still high on two instances, and the cause is not isolated. I escalate now, and I would have had the message drafted at T+15 when the first mitigations did not hold. The reasons are customer impact on an order endpoint, thirty minutes without a stable fix, and evidence pointing at a database and pool problem that L2 cannot fix alone.

Who and when. The escalation goes to the backend on-call (pool, retries, the query), the database or platform on-call (locks, sessions) and an incident commander for communication: one message first, then a bridge. I would escalate before T+30 if errors went above 10 %, more endpoints or tenants were affected, the SLA was at risk, or a mitigation made things worse.

What the escalation message contains

- **Timeline:** 14:02 alert, then each check and mitigation with time and effect.
- **Impact:** endpoints, instances, tenants, failed requests per minute, the enterprise customer's failure share.
- **Evidence:** pool 100/100/0, the 4.8 s query fingerprint, per-instance CPU, 5 s timeouts, retry counts, the misleading health check.
- **Tested and rejected:** each hypothesis with the data that rejected it.
- **Mitigations and effect:** for example 'draining instance X reduced errors from 7 % to 5 % for four minutes, then they returned'.
- **A specific ask:** inspect and kill sessions for fingerprint Y, say whether a plan change or lock explains 4.8 s, and give a go/no-go on raising the pool.
- **Severity and next update:** proposed SEV-2 and the time of the next customer update.

What would change the decision. If killing the query or capping retries had returned the error rate to baseline for fifteen minutes with CPU normal, I would open a ticket with the evidence for the owning team instead of running an incident. If only a non-critical endpoint were affected with no customer reports, I would hold a lower severity and escalate only if it persisted past the next check-in.

4. Three preventive improvements

Improvement	Improves	What exactly
Dependency-aware observability	Detection	Per-dependency metrics (pool use and queue wait, service and DB latency, retries, timeouts); alerts on pool > 80 % for 2 min and queue wait p95 > 500 ms; distributed tracing; a readiness probe that runs a cheap real query through the internal service
Resilience in the call path	Impact	Retry budget (≤ 10 % of requests, jittered backoff, idempotent only); DB timeout shorter than the API's (2 s < 5 s); circuit breaker; per-tenant concurrency limits and load shedding
Query and capacity hygiene	Recurrence	Alert on statements > 1 s; DB statement timeouts; fix the index or pin the plan; flag plan changes after statistics refreshes; pool size vs DB max connections review; runbook entry; post-incident review

Together these turn this incident from 'a customer called us' into 'an alert fired before the error rate moved', keep a slow query from becoming a 7 % error rate for everyone, and make sure the owning team fixes the query instead of on-call killing it again next month.